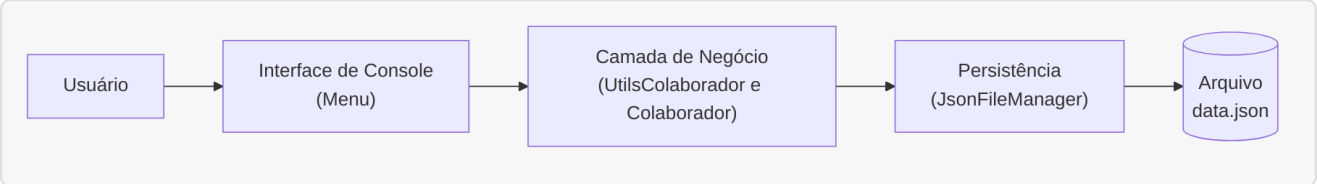
5. DOCUMENTO DE ARQUITETURA

Arquitetura adotada

Aplicação de console organizada em camadas simples: apresentação, negócio, persistência e armazenamento.



Tecnologias

Camada	Tecnologia
Front-end	Console (entrada via `Scanner`, saída via `System.out`)
Back-end	Java 15 ou superior, programação orientada a objetos
Biblioteca	json-simple 1.1.1
Banco	Arquivo JSON (`data.json`)
Build atual	`javac` e `java`, conforme o README
Testes e cobertura	JUnit 5 e JaCoCo (planejado; ver Estado da Infraestrutura de Testes)
Build planejado	Apache Maven (planejado; ver Estado da Infraestrutura de Testes)
Versionamento	Git e GitHub

Justificativas técnicas

Decisão	Justificativa
Java e POO	A herança representa naturalmente as categorias de colaborador e sustenta o requisito de extensibilidade (RNF004).
Console	Atende o escopo acadêmico com baixo custo e permite validar as entradas a cada passo (RNF002).
JSON em arquivo	Persistência leve, legível, sem servidor e sem configuração, suficiente para o volume do projeto e para o RNF003.
json-simple	Biblioteca pequena, sem dependências, que cobre a leitura e a escrita de JSON.

Organização dos componentes

Componente	Responsabilidade
`Main`	Ponto de entrada; inicia o menu
`Menu`	Exibe as opções e direciona cada uma ao tratamento correspondente
`UtilsColaborador`	Entradas do usuário, validações, cálculo de salário, relatórios e resumo
`Colaborador`, `ColaboradorComissionado`, `ColaboradorProducao`	Modelo de dados e conversão para JSON
`JsonFileManager`	Leitura, inclusão, alteração e remoção de registros no arquivo

Estratégia de persistência

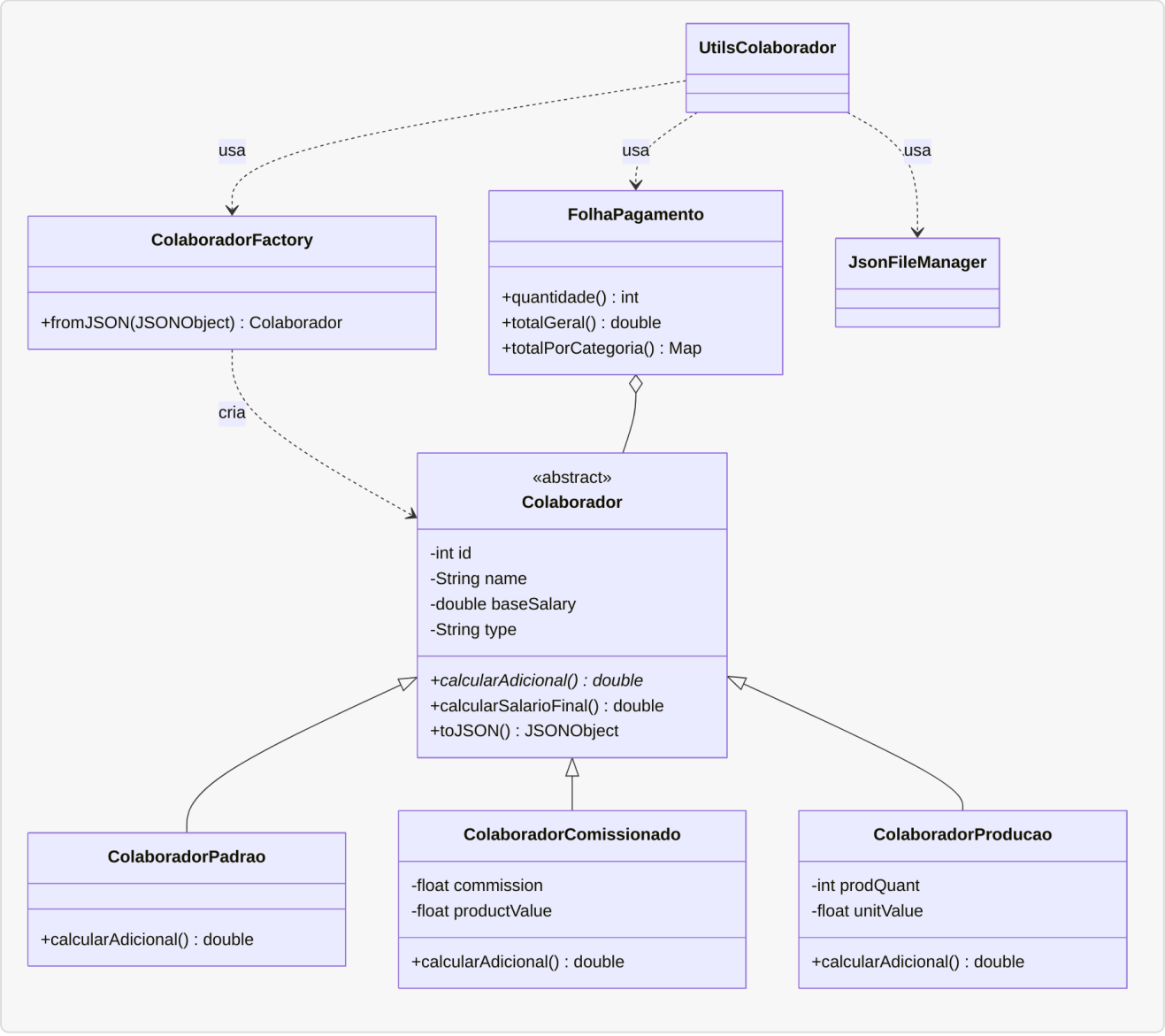
- Os colaboradores ficam em um array JSON no arquivo `data.json`, no diretório de execução.
- A cada operação o arquivo é lido por inteiro, alterado em memória e regravado.
- Arquivo inexistente ou vazio equivale a uma lista vazia.
- A matrícula (`id`) é a chave lógica e é verificada antes de cada inclusão.
- O salário final é armazenado junto ao registro e recalculado a cada alteração.

Exemplo de registro (colaborador comissionado):

```
{"id":1,"name":"Ana","type":"comissionado","baseSalary":2000.0,
 "finalSalary":2500.0,"commission":0.05,"productValue":10000.0}
```

Arquitetura Alvo do Cálculo

Para atender plenamente o RNF004, o cálculo do salário passa a ser uma operação de cada classe de colaborador. A classe base calcula o salário final como o salário base somado ao adicional, e cada categoria implementa apenas o seu adicional. O colaborador padrão passa a ter classe própria, e uma classe de folha concentra os totais.



Pontos do código que mudam

Ponto atual (`UtilsColaborador` e classes)	Situação hoje	Situação alvo
`criarColaborador`	`switch` por tipo calcula o salário final e instancia a classe	A fábrica instancia a classe do tipo; o salário final vem de `calcularSalarioFinal()`
`configuradorTipo`	`switch` converte a opção do menu em texto de tipo	Convertido em registro de tipos usado pela fábrica
`modificarColaborador` e `recalcularSalario`	`switch` e `if` por tipo alteram campos e recalculam	O colaborador é reconstruído pela fábrica e o recálculo é uma chamada a `calcularSalarioFinal()`
`imprimirColaborador` e `imprimirTodosColaboradores`	Exibição por tipo sobre o `JSONObject`	Exibição sobre objetos `Colaborador`
`imprimirRelatorio` e `imprimirResumo`	Adicionais e totais calculados por tipo	`FolhaPagamento` soma `calcularAdicional()` e `calcularSalarioFinal()`
`toJSON` de cada classe	Serialização por classe	Mantida; a fábrica faz o caminho inverso (`fromJSON`)

Hoje o sistema opera sobre os registros lidos do arquivo (`JSONObject`) e não reconstrói objetos `Colaborador`. Por isso a fábrica `fromJSON` é o primeiro passo da extração.

Plano de extração

Passo	Descrição	Responsável	Prazo
1	Tornar `Colaborador` abstrata, com `calcularAdicional()` abstrato e `calcularSalarioFinal()` igual a base mais adicional	Felipe de Miranda Borba	30/10/2026
2	Criar `ColaboradorPadrao` e implementar `calcularAdicional()` em `ColaboradorComissionado` e `ColaboradorProducao`	Felipe de Miranda Borba	30/10/2026
3	Criar `ColaboradorFactory.fromJSON` para reconstruir objetos a partir do arquivo	Felipe de Miranda Borba	02/11/2026
4	Criar `FolhaPagamento` e substituir os `switch` de criação, recálculo, relatório e resumo por chamadas polimórficas	Felipe de Miranda Borba	04/11/2026
5	Revisar por Pull Request, repetir CT002 a CT010 e reinspecionar REV-01	André Sergeievich Jaeger e João Gabriel da Cunha	05/11/2026

Limitações conhecidas

- Enquanto a extração não é concluída, o cálculo do salário e as validações permanecem concentrados em `UtilsColaborador`, e incluir um novo tipo exige alterar vários pontos desse arquivo. A inspeção REV-01 registra essa situação.
- Valores monetários em `float` podem gerar imprecisão de centavos; avaliar `double` ou `BigDecimal`.
- O build atual usa `javac`. O Maven, que gerenciaria as dependências (json-simple e JUnit) e executaria os testes e a medição de cobertura, está planejado e ainda não existe no repositório.